

INSTITUTO TECNOLÓGICO DE COSTA RICA
Campus Tecnológico Central Cartago
Escuela de Ingeniería en Computación

IC3101 – Arquitectura de Computadoras
II Semestre 2024

Reporte #6 - Semana 7 – Libro C - Capitulo 1
Introducción a C

Estudiante: Mauricio González Prendas – **Carné:** 2024143009

Profesor: Esteban Arias Méndez

Fecha de entrega: 09/09/2024

Bibliografía:

B. W. Kernighan and D. M. Ritchie, *The C programming language*, 2nd ed. Pearson, 2002, pp. 9–31.

Abstract—*This document covers essential C programming concepts, including function-based structure, variable types, constants, macros, and input/output operations. It discusses arrays, character strings, and the use of functions for code encapsulation and reuse. A practical example demonstrates temperature conversions, power calculations, and string processing, highlighting the application of functions, external variables, and loop constructs.*

I. Introducción al lenguaje C

El lenguaje de programación C se basa en funciones y variables, donde la función `main` sirve como punto de entrada principal del programa. Las funciones pueden invocar otras, ya sean definidas por el usuario o proporcionadas por bibliotecas estándar. Los comentarios, que se escriben entre `/*` y `*/`, permiten documentar el código sin influir en su ejecución.

II. Variables y tipos de datos

En C, las variables deben ser declaradas antes de su uso. Los tipos básicos incluyen `int` para enteros, `float` para números de punto flotante, y `char` para caracteres, entre otros como `short`, `long`, y `double`. La aritmética de punto flotante ofrece mayor precisión en cálculos, mientras que la aritmética entera puede causar truncamientos. La especificación de formato en la función `printf` controla la representación de estos datos.

III. Control de flujo y estructuras de repetición

El bucle `for` en C permite la repetición controlada de un bloque de código mediante una inicialización, una condición y un incremento, lo que lo hace eficiente para iteraciones cuando estos elementos están relacionados. Además, se emplean

estructuras de control como while e if para gestionar el flujo del programa y procesar datos de entrada.

IV. Constantes y macros

Las constantes simbólicas, definidas mediante #define, mejoran la legibilidad del código al evitar "números mágicos" y facilitar la modificación y mantenimiento al asignar nombres significativos a valores constantes.

V. Manejo de entrada y salida

Para leer y escribir caracteres, se utilizan las funciones getchar y putchar. El valor EOF indica el final de la entrada y se maneja usando el tipo int para garantizar una correcta identificación del final de archivo. Las funciones de entrada y salida son esenciales para el conteo de caracteres, líneas y palabras.

VI. Arreglos y cadenas de caracteres

Los arreglos permiten almacenar múltiples valores del mismo tipo en una sola variable, accediendo a sus elementos mediante índices. Los arreglos de caracteres, o cadenas, se terminan con el carácter nulo \0, que marca el final de la cadena y es utilizado por las funciones para determinar su longitud y contenido.

VII. Funciones y argumentos

Las funciones en C encapsulan instrucciones para tareas específicas, permitiendo la reutilización del código. Cada función se define con un tipo de retorno, nombre y parámetros. Los argumentos se pasan por valor, proporcionando copias temporales de los valores, mientras que los punteros permiten modificar variables externas. Las variables externas, definidas fuera de las funciones, mantienen sus valores a lo largo de las llamadas y simplifican la comunicación entre funciones, aunque su uso excesivo puede complicar el mantenimiento del código.

VIII. Declaración y alcance de variables

Las variables automáticas, definidas dentro de una función, no conservan sus valores entre llamadas. En contraste, las variables externas conservan sus valores y pueden ser accedidas en múltiples funciones. La correcta utilización de estas variables es clave para la organización y la mantenibilidad del código.

IX. Ejemplo práctico

Este ejemplo ilustra el uso de los conceptos discutidos en los apartados anteriores mediante un programa en C que realiza conversiones de temperatura, calcula potencias de números, y maneja cadenas de texto.

```
#include <stdio.h>
```

```

#define LOWER 0
#define UPPER 100
#define STEP 10

// Declaración de funciones
int power(int base, int n);
void copy(char to[], char from[]);
int getline(char line[], int maxline);

int max; // Variable externa
char line[1000]; // Variable externa
char longest[1000]; // Variable externa

int main() {
    int fahr;
    float celsius;

    // Imprimir tabla de conversiones de Fahrenheit a Celsius
    for (fahr = LOWER; fahr <= UPPER; fahr += STEP) {
        celsius = (5.0 / 9.0) * (fahr - 32);
        printf("%3d %6.1f\n", fahr, celsius);
    }

    // Ejemplo de uso de funciones
    int i;
    printf("\nPotencias de 2:\n");
    for (i = 0; i < 10; ++i) {
        printf("2^%d = %d\n", i, power(2, i));
    }

    // Encontrar la línea más larga de entrada
    int len;
    max = 0;
    while ((len = getline(line, 1000)) > 0) {
        if (len > max) {
            max = len;
            copy(longest, line);
        }
    }

    if (max > 0) {
        printf("\nLínea más larga:\n%s", longest);
    }

    return 0;
}

// Función que eleva un número a una potencia

```

```

int power(int base, int n) {
    int p = 1;
    for (int i = 1; i <= n; ++i) {
        p *= base;
    }
    return p;
}

// Función que lee una línea de entrada
int getline(char s[], int lim) {
    int c, i;
    for (i = 0; i < lim - 1 && (c = getchar()) != EOF && c != '\n';
        ++i) {
        s[i] = c;
    }
    if (c == '\n') {
        s[i] = c;
        ++i;
    }
    s[i] = '\0';
    return i;
}

// Función que copia una cadena a otra
void copy(char to[], char from[]) {
    int i = 0;
    while ((to[i] = from[i]) != '\0') {
        ++i;
    }
}

```

Explicación:

1. Inclusión de bibliotecas y definiciones de constantes:

- `#include <stdio.h>`: Incluye la biblioteca estándar de entrada/salida.
- `#define LOWER 0, #define UPPER 100, #define STEP 10`: Define constantes simbólicas para los límites y el paso en la tabla de conversiones.

2. Declaración de variables externas:

- `int max;, char line[1000];, char longest[1000];`: Declaración de variables externas que son accesibles desde múltiples funciones.

3. Función main:

- **Tabla de Conversión:** Usa un bucle for para imprimir una tabla de conversiones de Fahrenheit a Celsius.

- **Uso de Funciones:** Llama a la función `power` para calcular y mostrar potencias de 2.
- **Lectura y Procesamiento de Entrada:** Usa `getline` para leer líneas de entrada y `copy` para guardar la línea más larga.

4. Función `power`:

- Calcula la potencia de un número base elevado a un exponente entero `n`.

5. Función `getline`:

- Lee una línea de entrada hasta un límite especificado y la almacena en el arreglo `s`. Retorna la longitud de la línea leída.

6. Función `copy`:

- Copia una cadena de caracteres desde `from` a `to`.

Aspectos claves:

- **Variables externas:** `max`, `line`, y `longest` son accesibles en toda la unidad de traducción.
- **Funciones:** la declaración de funciones `power`, `getline`, y `copy` ayuda a organizar y reutilizar el código.
- **Uso de constantes simbólicas:** `lower`, `upper`, y `step` hacen que el código sea más legible y fácil de modificar.
- **Bucles y condicionales:** muestran cómo iterar sobre rangos y manejar condiciones en c.